

EvidenceMalGraph: An evidence-grounded modular framework for Android malware family attribution and campaign analysis

Shishuai Yang^a, Xiaoyu Li^a, Kailun Yan^b, Chao Wang^a, Weixiang Ren^{a,*},
Yangrong Chen^a, Kebao Chen^a

^a Zhengzhou University of Aeronautics, Zhengzhou, Henan, China
^b Tsinghua University, Beijing, China

ARTICLE INFO

Keywords:

Android malware
Family attribution
Modular evidence analysis
Campaign analysis

ABSTRACT

Android malware analysis should go beyond distinguishing malicious apps from benign ones. After detecting a malicious APK, analysts still need to attribute its family, inspect related samples, and discover cross-family relationships. This task is difficult because variants within the same family may change over time, while different families may reuse similar SDKs, repackaging templates, infrastructure, texts, or behaviors. In this work, we propose EVIDENCEMALGRAPH, an evidence-grounded modular framework for Android malware family attribution and Campaign analysis. EVIDENCEMALGRAPH organizes static APK evidence through five specialized analyzers covering certificates, package structure, text, infrastructure, and behavior. Their evidence is fused to attribute test APKs against labeled training APKs. Rather than discarding source information after fusion, EVIDENCEMALGRAPH retains the five similarity components on graph edges and aggregates the same analyzer outputs into citable Campaign evidence. Family attribution, Campaign discovery, and reporting therefore remain grounded in the same APK-derived records. The system then constructs an APK similarity graph to identify Campaign candidates and generates reports constrained by a local evidence catalog. We evaluate EVIDENCEMALGRAPH on the MalRadardataset. Of 4383 APKs, 4302 are parsed. Under the known-family external holdout setting, EVIDENCEMALGRAPH achieves 90.76% Accuracy and 73.21% Macro F1, outperforming all single-source variants and two same-protocol baselines: a Drebin-inspired SVM and a DeepAMD-style MLP. Its gains over Text-only remain positive across 10,000 bootstrap resamples, and sensitivity analyses produce consistent results. It also identifies 244 Campaign candidates with 97.12% multi-sample Campaign coverage and 97.68% edge family homogeneity. The generated reports achieve 100% on all verification metrics. These results show that EVIDENCEMALGRAPH supports malware family attribution, sample relationship discovery, and reproducible evidence-grounded reporting.

1. Introduction

Android has become one of the most widely used mobile operating systems (StatCounter, 2026), and Android apps are distributed at massive scale through official (Google Play) and third-party markets. The openness of the Android ecosystem greatly facilitates app development and distribution, but it also creates favorable conditions for malware authors. Malicious APKs can be rapidly repackaged, renamed, redistributed, and evolved into many variants. As a result, security analysts often need to process large numbers of malicious samples and determine whether newly observed APKs are related to known malware families, shared malicious components, or broader attack activities.

Traditional Android malware analysis mainly focuses on malware detection, where the primary goal is to distinguish malicious apps from

benign apps (Arp et al., 2014). However, detection is only the first step in many security analysis workflows. After a sample is flagged as malicious, analysts still need to determine which known malware family it is related to and whether it shares evidence with other samples. At this stage, analysts are more concerned with family attribution and relationship analysis. Specifically, they need to know which known malware family a sample is closest to, which pieces of evidence support this attribution, whether multiple samples reuse similar code or behavioral capabilities, and whether samples with different family labels still share suspicious relationships. Therefore, Android malware analysis requires not only classification results, but also evidence-grounded explanations and sample-level relationship discovery.

This task is challenging for several reasons. First, Android malware families usually contain many variants. Samples from the same family

* Corresponding author.

E-mail addresses: shishuai@zua.edu.cn (S. Yang), csxyli@zua.edu.cn (X. Li), kailun@mail.tsinghua.edu.cn (K. Yan), zh_cw@zua.edu.cn (C. Wang), renweixiang@zua.edu.cn (W. Ren), yrcen@zua.edu.cn (Y. Chen), M25080854025@zua.edu.cn (K. Chen).

<https://doi.org/10.1016/j.eswa.2026.134346>

Received 1 July 2026; Received in revised form 25 August 2026; Accepted 7 September 2026

Available online 8 September 2026

0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

may change their package names, certificates, resource texts, URLs, or permissions across versions (Mariconti et al., 2017). Second, different families may reuse public SDKs, repackaging templates, shared infrastructure, deceptive texts, or similar malicious capabilities. This makes a single evidence source unreliable (Li et al., 2017; Zhang et al., 2014). Third, malware family distributions are usually long-tailed (Wang et al., 2022). A few families contain many samples, while many other families have only a small number of APKs. In this case, overall Accuracy alone cannot fully reflect whether a method works well for smaller families. Finally, family labels themselves may contain aliases, inconsistent granularity, and labeling errors (Sebastián & Caballero, 2020). Therefore, directly treating a cluster or graph community as a confirmed malware family or a real-world attack Campaign may lead to over-interpretation.

Our Work. In this work, we propose EVIDENCEMALGRAPH, an evidence-grounded modular framework for Android malware family attribution and Campaign analysis. Instead of merging all static features into a single hard-to-inspect representation, EVIDENCEMALGRAPH organizes Android malware evidence through five specialized analyzers for certificates, package structure, text, infrastructure, and behavior. Each analyzer deterministically extracts and normalizes one type of evidence and produces an independent similarity signal. These signals are then fused to support family attribution by comparing each test APK only with labeled APKs in the training set. After family attribution, the system constructs an APK similarity graph and applies community detection to discover Campaign candidates. For each Campaign candidate, the system further builds a local evidence catalog and generates an evidence-grounded report. Multi-sample Campaign reports are written by deepseek-v4-flash using only privacy-preserving aggregated evidence, while singleton reports are generated by a deterministic template.

A central feature of this workflow is that the evidence is not discarded after a family label has been predicted. When two APKs are compared, EVIDENCEMALGRAPH calculates separate similarities for their certificates, package structures, texts, infrastructure, and behaviors, and then combines them into an overall score. This score is used to find the nearest training samples and to decide whether the two APKs should be connected in the graph. The five component scores remain on the graph edge, so analysts can see which evidence sources produced the connection. For each Campaign candidate, the corresponding analyzer outputs are also summarized in a local evidence catalog that supports the report claims.

Prior attribution methods mainly evaluate the final detection or family label (Hou et al., 2017; Karbab et al., 2018; Narayanan et al., 2018), while clustering methods focus on the groups formed from behavioral, code, or payload similarity (Jang et al., 2011; Li et al., 2017; Rieck et al., 2008). EVIDENCEMALGRAPH instead preserves a verifiable evidence trail across all three outputs: a family prediction can be traced to its nearest training reference, every Campaign graph edge retains the five similarity components that created the link, and every report claim cites an entry in the local evidence catalog. We therefore evaluate family attribution, evidence contribution, graph-based Campaign analysis, and evidence-grounded reporting through the following four research questions:

- ⇒ **RQ1:** How well do individual evidence sources and the complete EvidenceMalGraph framework attribute external-test APKs to families represented in the training set?
- ⇒ **RQ2:** How does family attribution performance change when each evidence source is removed, and which sources contribute most to the fused result?
- ⇒ **RQ3:** How well do the graph communities group APKs with consistent static evidence and family labels, and can they surface cross-family relationships for further investigation?
- ⇒ **RQ4:** Can the analysis reports automatically generated for Campaigns provide clear evidence, complete content, and reproducible verification?

To answer these research questions, we evaluate EVIDENCEMALGRAPH on the MalRadar dataset. The original sample list contains 4383 APKs covering 171 training families. During static analysis, 4302 samples are successfully parsed and used in subsequent experiments. Under the external holdout setting, the successfully parsed training set contains 3436 APKs, and the test set contains 866 APKs covering 118 families. To avoid test-set leakage, model and parameter selection, fusion learning, and text-vocabulary learning are performed only on the subtrain and validation subsets. The external test set is used only once for the final evaluation.

The results show that EVIDENCEMALGRAPH achieves 90.76% Accuracy and 73.21% Macro F1 on the external test set, outperforming the evaluated single-source variants and supervised family classifiers. Compared with the two supervised family classifiers, it improves Accuracy by 2.31–3.12 percentage points and Macro F1 by 3.30–7.08 points. Across 10,000 bootstrap resamples, the 95% paired-gain intervals over Text-only are 2.42–4.85 points for Accuracy and 2.41–5.14 points for Macro F1. Fusion, voting, ablation, and graph-construction sensitivity tests preserve the main conclusions. The graph-based analysis identifies 244 Campaign candidates from 4302 successfully analyzed APKs, with 97.12% multi-sample Campaign coverage and 97.68% edge family homogeneity. In addition, all generated Campaign reports achieve 100% on schema compliance, source reproducibility, citation validity, evidence coverage, and Markdown completeness.

Contributions. Here we list the main contributions of this paper:

- **Multi-source evidence for family attribution.** EVIDENCEMALGRAPH compares each test APK with labeled training samples from five perspectives: certificate, package structure, text, infrastructure, and behavior. The five similarity scores are combined to select the nearest references but remain available separately. This makes it possible to determine which evidence sources drive a family match rather than relying on the fused score alone.
- **Systematic evaluation of family attribution and evidence contribution.** Under the external protocol, EVIDENCEMALGRAPH achieves 90.76% Accuracy and 73.21% Macro F1, exceeding Text-only, the strongest single source, by 3.58 and 4.00 percentage points and outperforming both same-protocol supervised baselines, the Drebin-inspired SVM and DeepAMD-style MLP, on both metrics. On validation, the knowledge-guided weights lead all four fusion strategies with 93.67% Accuracy and 80.42% Macro F1, while neither learned strategy improves consistently across validation and test.
- **Graph-based Campaign candidate discovery.** We extend family attribution to APK relationship analysis by constructing a similarity graph based on multi-source static evidence. The system identifies 244 Campaign candidates from 4302 successfully analyzed APKs, including 120 multi-sample candidates and 124 singleton candidates. The multi-sample coverage reaches 97.12%, and the edge family homogeneity reaches 97.68%. These values indicate broad graph coverage and high agreement with existing family labels. The graph also surfaces cross-family candidates for further investigation.
- **Reproducible and evidence-grounded report generation.** We design a report-generation mechanism that constrains Large Language Model (LLM) outputs with a local evidence catalog. Each generated claim must cite a valid evidence ID, and invalid citations are filtered before saving. The generated Campaign reports achieve 100% on schema compliance, source reproducibility, citation validity, evidence coverage, and Markdown completeness.

Overall, this work provides an analyst-facing framework for Android malware family attribution and Campaign analysis. Its methodological contribution is to preserve the five source-specific similarities after fusion and reuse them across family attribution, graph interpretation, and report verification. At the engineering level, EVIDENCEMALGRAPH is designed for use in practical security workflows. Parallel APK parsing, candidate-pair retrieval, and immediate per-APK result storage

support dataset-scale analysis. Saved analyzer records and evidence-backed graph and report outputs allow analysts to inspect how each result was produced and reproduce it from the stored intermediate evidence. By connecting APK parsing, family attribution, Campaign discovery, and report generation, the system provides a complete path from APK input to verifiable analysis output.

Roadmap. The rest of this paper is organized as follows. [Section 2](#) introduces the background of Android malware family attribution, static evidence, and similarity-graph-based Campaign candidates. [Section 3](#) presents the design of EVIDENCEMALGRAPH. [Section 4](#) details the dataset and evaluation protocol, compared methods, parameter-selection and sensitivity protocols, evaluation metrics, and bootstrap analysis. [Section 5](#) reports the experimental results according to the four research questions. [Section 6](#) discusses threats to validity. [Section 7](#) reviews related work, and [Section 8](#) concludes this paper.

2. Background

This section introduces the background concepts needed to understand EVIDENCEMALGRAPH, including static evidence used in APK analysis, Android malware family attribution, and similarity-graph-based Campaign candidate discovery.

2.1. Static evidence in Android malware analysis

Android APKs contain multiple types of static evidence that can be extracted without executing the app. Common evidence sources include signing certificates ([Google, 2026f](#)), package names, class structures, strings, URLs, domain names, permissions ([Google, 2026e](#)), components ([Google, 2025a](#)), and Android API calls. These features describe different aspects of an APK and provide complementary views for malware analysis.

- Signing certificates can reveal reuse relationships among APKs. If multiple samples are signed with the same certificate, they may be produced by the same developer, generated by the same toolchain, or distributed through the same repackaging process. However, certificate evidence is not always reliable. Some malware samples use public test certificates, and some samples may be batch re-signed by third-party packaging tools. Therefore, certificate reuse is a useful clue but should not be treated as direct proof of a shared family.
- Package names and class structures describe the namespace and code organization of an APK. They can capture code-template reuse, repackaging traces, and stable naming conventions across variants. However, they are also affected by package-name reuse, obfuscation, impersonation, and repackaging. A shared package prefix may indicate a real relationship, but it may also come from copied templates or automatically generated code.
- Textual evidence includes DEX strings ([Google, 2025b](#)) and resource texts. In Android malware, text can be highly informative because many samples contain deceptive messages, fake update prompts, login phrases, phishing keywords, gambling vocabulary, or other social-engineering content. These strings can reveal how a sample attempts to interact with or deceive users. However, textual evidence may also be noisy. Different families may reuse the same advertising templates, public SDK strings, or common fraud phrases.
- Infrastructure evidence includes URLs, domain names, and API endpoints extracted from APK strings. Such evidence can indicate backend reuse and operational similarity. It is especially useful when multiple samples contact the same suspicious domain or share similar endpoint patterns. However, infrastructure evidence may be missing from some APKs, and families may change server-side addresses across versions. Public cloud, analytics, and advertising domains can also introduce noisy similarities.
- Behavior-related evidence can be inferred from permissions, components, and API calls. Permissions describe the capabilities requested

by an APK, while components and API calls provide more direct clues about how these capabilities may be used. For example, SMS-related APIs ([Google, 2026g](#)), device-administration components, overlay windows ([Google, 2026d](#)), accessibility services ([Google, 2026a](#)), and dynamic code loading can indicate sensitive or malicious behaviors. Nevertheless, static behavior evidence still has limitations. It may not fully capture dynamically loaded code, encrypted strings, reflection, native code, or conditionally triggered logic.

2.2. Android malware family attribution

Android malware family attribution aims to determine which known malware family a malicious APK is most closely related to ([Microsoft, 2025](#)). This task is different from malware detection. Malware detection focuses on distinguishing malicious apps from benign apps ([Arp et al., 2014](#)), while family attribution is performed after an app has already been considered malicious. Its goal is to compare a target APK with known malicious samples and assign it to the most likely family according to shared evidence. Family attribution is useful for several downstream security tasks. First, it helps analysts triage large numbers of malicious samples by grouping newly observed APKs with previously known families. Second, it supports threat understanding by revealing whether a new sample is likely to reuse known code, behaviors, infrastructure, or social-engineering content. Third, it enables similarity-based retrieval, where analysts can inspect samples that are close to a query APK. Finally, family attribution provides a basis for broader relationship analysis, because samples assigned to the same or nearby families may share development practices, repackaging chains, or operational resources.

However, Android malware family attribution is not a trivial classification task. Malware families often contain many variants, and different families may reuse similar components, SDKs, certificates, package structures, or deceptive texts. At the same time, the number of samples is usually imbalanced across families. Some families contain many samples, while many others are represented by only a few APKs. These characteristics make it necessary to use multiple sources of evidence rather than relying on a single feature type.

2.3. Malware similarity graphs and campaign candidates

A malware similarity graph represents APKs as nodes and evidence-based similarities as edges ([Frenklach et al., 2021](#)). In this graph, two APKs are connected when they share sufficient evidence across one or more feature types. The edge weight can reflect the strength of their similarity, while edge attributes can record which evidence categories support the connection. This graph-based representation is useful because malware relationships are often not limited to one-to-one family predictions. Instead, samples may form groups with shared code templates, behaviors, infrastructure, or deceptive content.

Graph structures can support malware cluster discovery ([Blondel et al., 2008](#)). If a group of APKs are densely connected, they may represent variants of the same family, samples generated from a shared repackaging chain, or apps using similar malicious components. Community detection can therefore help identify sample groups that deserve further investigation. Compared with isolated pairwise similarity scores, graph communities provide a higher-level view of how malicious APKs are organized in the dataset. Similarity graphs can also reveal cross-family relationships. Some APKs may have different family labels but still share strong static evidence. Such cases are important for security analysis because they may indicate shared malicious components, family evolution, infrastructure reuse, common fraud templates, or inconsistencies in the existing labeling system. However, a cross-family connection should not be directly interpreted as proof that the samples are controlled by the same attacker.

For this reason, this work uses the term Campaign candidate. A Campaign candidate refers to a group of APKs discovered from static evi-

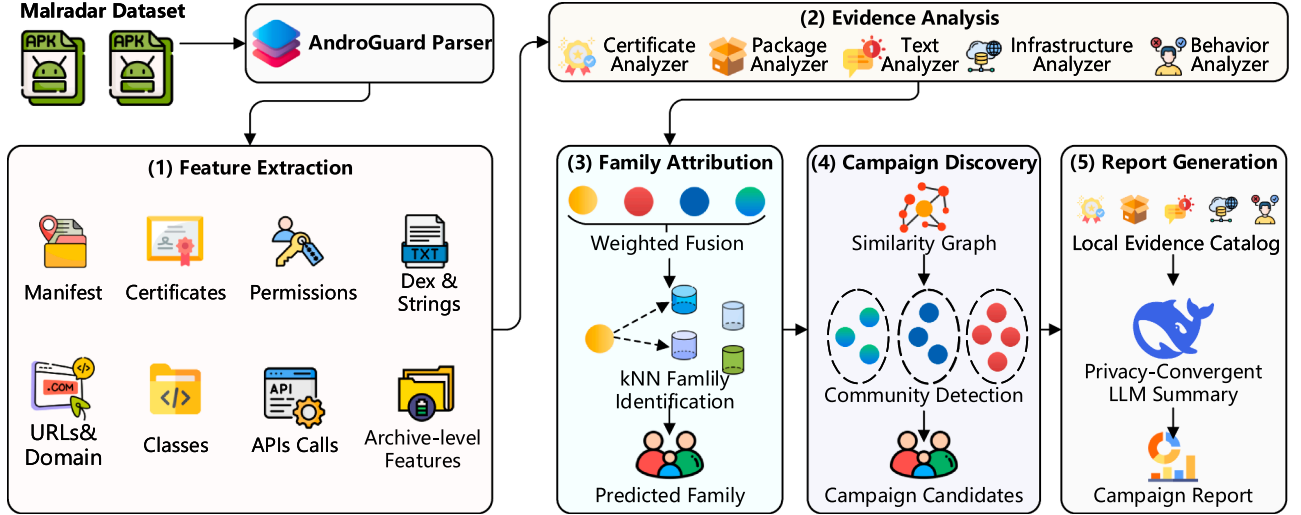


Fig. 1. Overview of the EVIDENCEMALGRAPH framework.

dence similarity that is worth further investigation. It is not a confirmed real-world attack Campaign. Confirming a real Campaign usually requires additional external intelligence, such as attacker identity, infrastructure ownership, propagation timeline, distribution channel, or incident context. Therefore, Campaign candidates in this work are best understood as evidence-based analysis hypotheses that can support triage, relationship exploration, and subsequent manual investigation.

3. Methodology

EVIDENCEMALGRAPH consists of five stages, as shown in the framework diagram in Fig. 1:

- **Feature Extraction.** The system uses Androguard (Github, 2017) to statically parse APKs and independently stores the intermediate analysis results for each APK.
- **Evidence Analysis.** Based on the parsed results, five specialized analyzers extract comparable evidence features from the perspectives of certificates, package structure, text, infrastructure, and behavior.
- **Family Attribution.** The system constructs a multi-source evidence representation for each test APK and computes its similarity to APKs in the training set with known family labels. It then performs neighborhood voting or weighted fusion based on the most similar training samples to obtain the family prediction.
- **Campaign Discovery.** After family attribution, the system constructs a similarity graph based on evidence similarity among APKs and discovers Campaign candidates from the graph.
- **Report Generation.** For each Campaign, the system constructs privacy-preserving aggregated evidence from the local evidence catalog and invokes the LLM deepseek-v4-flash (DeepSeek-AI, 2026) to generate verifiable Campaign reports with evidence citations.

The five modules share a common evidence flow. Each analyzer produces a source-specific record for an APK. When two APKs are compared, the system calculates a separate similarity score for each of the five evidence sources. The scalar fusion score is used for reference ranking and graph-edge selection, while the components remain available for edge interpretation. Campaign reports are built from evidence catalogs aggregated from the same analyzer records. In this way, evidence extracted at the beginning remains traceable through family decisions, graph connections, and report statements. Sections 3.1 to 3.5 describe how this evidence flow is implemented across the five modules.

3.1. Feature extraction

The system processes APKs through static analysis. It does not execute the apps themselves or connect to any external addresses found in the samples. For each APK, the system directly reads the Manifest (Google, 2026b), DEX files, resource files (Google, 2026c), and signature information from the APK, and organizes these raw parsing results into a unified feature set that can be used by subsequent evidence analyzers. Given an APK (x), its static parsing result is represented as:

$$F(x) = \{M, C, P, S, U, D, A, R\} \quad (1)$$

where (M) denotes Manifest and component information, (C) denotes signing certificates, (P) denotes permissions, (S) denotes DEX and resource strings, (U) denotes URLs and domain names, (D) denotes class structures, (A) denotes Android API calls, and (R) denotes archive-level features such as compression methods, the number of DEX files, and native libraries.

In the implementation, the system uses Androguard 4.1.4 (Github, 2017) for static APK parsing. Among the 4383 input APKs, 4302 samples were successfully parsed and included in subsequent experiments. The remaining 81 samples were excluded from further analysis due to file format problems or instruction parsing failures.

3.2. Evidence analysis

This stage analyzes the static features extracted in the previous step and converts them into evidence representations that can be compared across APKs. It contains five specialized analyzers, each responsible for organizing one evidence source: certificate, package, text, infrastructure, or behavior. Their outputs provide the comparable evidence used by the subsequent fusion module to determine which aspects two APKs share and how strong each similarity is.

3.2.1. Certificate analyzer

The Certificate Analyzer captures signing-certificate reuse relationships among APKs. This analyzer records the SHA-256 hash, SHA-1 hash, issuer, subject, and validity period of each signing certificate. If two APKs share the same non-empty certificate SHA-256 hash, their certificate similarity is set to 1. Otherwise, it is set to 0. Identical certificates do not necessarily indicate that two APKs belong to the same malware family, because some APKs may use default test certificates or may be re-signed by the same batch packaging tool. Therefore, certificate evidence is assigned a weight of only 0.10 in the fusion process.

3.2.2. Package analyzer

The Package Analyzer captures namespace and code-organization similarity. It compares exact package names and shared prefixes, and hashes class-name and package-path fragments into a 64-dimensional vector whose cosine similarity summarizes structural overlap. Exact names receive priority, while prefix and vector similarity support relationships among renamed or repackaged variants.

3.2.3. Text analyzer

The Text Analyzer combines rule matching with vocabulary learned from training data. After filtering class names, method signatures, API names, and library paths, it matches themes such as phishing, SMS abuse, fake updates, gambling, and credential theft. TF-IDF features complement these rules by capturing family-specific terms. For token t , IDF is defined as:

$$\text{IDF}(t) = \log \frac{1 + N}{1 + \text{DF}(t)} + 1. \quad (2)$$

Here, N is the number of training documents and $\text{DF}(t)$ is the number containing t . Terms are ranked by IDF, concentration in their dominant family, and within-family recall. The vocabulary is learned from training data and fixed before test APKs are transformed, preventing test text from influencing its construction.

3.2.4. Infrastructure analyzer

The Infrastructure Analyzer focuses on backend resources that samples may connect to. The system extracts URLs, domain names, and API endpoints with query parameters removed from strings, and measures infrastructure similarity between two APKs using the Jaccard similarity (IBM, 2026) of their domain sets. Advertising, analytics, and public cloud domains are widely reused across different apps. Therefore, infrastructure evidence is assigned a weight of only 0.05. This limitation is also retained in Campaign reports: sharing public services does not imply a common operator.

3.2.5. Behavior analyzer

The Behavior Analyzer maps permissions, API calls, and components to 26 categories of sensitive behaviors. These behaviors include SMS abuse, accessibility abuse, overlay attacks, notification reading, automatic installation, device administration, contact and location collection, audio and video recording, account and file collection, screen and clipboard capture, app discovery, VPN interception, persistence, dynamic code loading, command execution, and anti-analysis.

Permissions only indicate that an APK has a certain capability. Therefore, when a permission appears alone, its confidence score is set to 0.35. API calls or component declarations provide more direct implementation evidence. When both permissions and implementation evidence are present, the behavior confidence score is set to 1.0. For behaviors that depend on specific permissions, such as SMS sending and device administration, the system does not determine the existence of the behavior based only on ordinary API calls. Instead, it requires the corresponding permission, component, or more explicit implementation evidence to appear together. To avoid the impact of low-confidence behaviors on sample similarity computation, the system treats only behaviors with confidence scores no lower than 0.70 as strong behaviors, which are used for graph construction and Campaign reports.

3.3. Family attribution

After the five evidence analyzers produce source-specific evidence records, this stage computes one similarity score for each source and combines the scores for family attribution. Test APKs are compared only with training APKs whose family labels are known. For APKs i and j , the system retains the five similarity scores as a five-dimensional evidence vector:

$$r(i, j) = [s_c(i, j), s_p(i, j), s_t(i, j), s_n(i, j), s_b(i, j)]^T. \quad (3)$$

The superscript T denotes transposition, indicating that the five similarity components are arranged as a column vector. where s_c , s_p , s_t , s_n , and s_b denote certificate, package-structure, text, infrastructure, and behavior similarity, respectively. All five scores lie between 0 and 1, but each analyzer defines its score according to its own evidence. For example, a certificate score of 1 means that two APKs have the same certificate, while a text score of 1 means that they reach the strongest match under the text-similarity function. A text score of 0.5 and a behavior score of 0.5 therefore do not necessarily represent equally strong evidence. The system does not compare these raw scores to select the analyzer with the largest value. Instead, it applies the same weighted fusion function to every candidate APK pair and ranks the reference APKs by the resulting fused score. Family attribution therefore compares APK pairs under one consistent fusion rule rather than comparing scores from different analyzers directly. The system fuses these five similarities into a unified similarity score:

$$S(i, j) = 0.10s_c + 0.20s_p + 0.30s_t + 0.05s_n + 0.35s_b. \quad (4)$$

The weights encode the expected reliability of the five evidence sources. Behavior (0.35) and text (0.30) receive the largest weights because they describe what an APK is prepared to do and the family-specific content it presents. Package structure receives a moderate weight (0.20) because namespaces and code organization can survive across variants but may change after repackaging or obfuscation. Certificate (0.10) and infrastructure (0.05) receive smaller weights because APKs can be re-signed, public certificates can be reused, network indicators may be absent, and unrelated apps may contact the same public service. These values were fixed before the external-test evaluation. Section 4 compares this knowledge-guided score with uniform fusion and two fusion functions learned without using the external test set.

For family attribution, $S(i, j)$ orders labeled training APKs by their overall similarity to a test APK, and the nearest labels support the prediction. The same score is later used for graph thresholding, while $r(i, j)$ remains attached to retained edges for source-level inspection. Text similarity combines training-set TF-IDF cosine similarity with rule-based similarity. Behavior similarity combines strong-behavior overlap with confidence-map similarity. To avoid exhaustive $O(n^2)$ comparison, source-specific indexes and locality-sensitive hashing first retrieve candidate pairs, after which $r(i, j)$ and $S(i, j)$ are computed only for those pairs.

For test query q , only training APKs can serve as references. A top neighbor is used alone when its score is at least 0.50 or exceeds the second score by at least 0.02. Otherwise, at most three neighbors vote. For candidate family y , the voting score is:

$$V(y | q) = \sum_{r \in \mathcal{N}(q), y_r = y} S(q, r). \quad (5)$$

Here, $\mathcal{N}(q)$ is the selected neighbor set and y_r is reference r 's family. Each selected reference contributes its fused similarity, and contributions from references with the same family are added together. The family with the highest $V(y | q)$ is returned. To assess whether this decision rule depends on its preset values, Section 4 varies the maximum neighbor count, score gap, and high-similarity cutoff.

3.4. Campaign discovery

After family attribution, EVIDENCEMALGRAPH further uses the fused similarity $S(i, j)$ obtained in the previous section to construct an APK similarity graph, which is used to discover groups of samples with potentially shared evidence characteristics. The system constructs an undirected weighted graph $G = (V, E)$, where each node represents an APK. When the fused similarity between two APKs satisfies $S(i, j) \geq 0.45$, the system adds an edge between them and stores the overall similarity as well as the five evidence components, including certificate, package structure, text, infrastructure, and behavior, as edge attributes. The sensitivity analysis compares this cutoff with nearby fixed values, a thresh-

old estimated from the observed similarity distribution, and top- k neighbor graphs.

On this similarity graph, the system applies the Louvain algorithm (Blondel et al., 2008) for community detection. The Louvain algorithm assigns more densely connected nodes to the same community, making intra-community connections relatively dense and inter-community connections relatively sparse. The system uses a fixed random seed of 42 and a resolution of 1 to ensure stable community detection results across repeated runs. Following the definition in Section 2.3, each detected community is stored as a Campaign candidate. For each candidate, the system records its family composition, edge evidence, evidence-category coverage, and limitation notes. For cross-family communities, these notes preserve possible explanations such as shared malicious components, repackaging chains, infrastructure reuse, public SDKs, shared certificates, or label noise. These records are later used for Campaign-level analysis and evidence-grounded report generation.

3.5. Report generation

After Campaign discovery, EVIDENCEMALGRAPH further generates verifiable analysis reports for each Campaign candidate. To prevent the report content from drifting away from local evidence, the system first constructs a local evidence catalog for each Campaign. Each catalog entry contains a unique `evidence_id`, an evidence category, a metric value, a sample count, and a coverage ratio, which are used to constrain factual statements in the generated report.

For Campaigns containing at least two APKs, the system sends a privacy-preserving aggregated payload to `deepseek-v4-flash`. This payload contains only the number of APKs, family distribution, behavior categories, text categories, and aggregated counts. APK hashes, domain names, certificate identifiers, package names, and single-sample identities are retained in the local environment and are not sent to the LLM.

`deepseek-v4-flash` is responsible for generating the report title, overall assessment, claims, and limitations. Each claim must cite a valid `evidence_id` from the local evidence catalog. The system automatically removes invalid citations and uses a deterministic template to complement evidence categories not covered by the model. For Campaigns containing only a single APK, the system does not invoke `deepseek-v4-flash` and instead directly generates the report using a deterministic template. The final report also stores a source summary, allowing the evaluator to recompute and verify whether the report is consistent with the evidence catalog of the current Campaign.

4. Experimental design

This section describes how EVIDENCEMALGRAPH is evaluated. It presents the dataset and evaluation protocol, the compared methods and sensitivity analyses, the evaluation metrics, and the bootstrap procedure. These experiments assess family attribution, evidence contribution, Campaign-candidate discovery, and report verification while keeping the external test set separate from model and parameter selection.

4.1. Dataset and evaluation protocol

This subsection introduces the dataset and label sources, defines the outer temporal and internal validation splits, and describes the controls used to prevent evaluation leakage.

Dataset and Label Sources. The experiment uses the MaIRadar dataset (Wang et al., 2022) and the `sample-info.csv` metadata provided with the samples. MaIRadar is a dataset for Android malware research. Its samples are collected from mobile security reports released by multiple security vendors and are organized based on Indicators of Compromise (IoC) information and expert analysis results. The original sample list contains 4383 APKs covering 171 training families. During

static analysis, 4302 samples were successfully parsed, resulting in a success rate of 98.15%. Of the 4383 APKs, 3967 use the `report_family` label supplied with MaIRadar. The remaining 416 use `AVClass_family` because `report_family` is unavailable. These fields do not provide two independent labels for the same APKs in this corpus, and this study does not manually relabel or adjudicate the samples.

Data Splits and Evaluation Controls. The main experiment adopts a family-stratified temporal split with a fixed random seed of 42. Specifically, for each family with at least two samples, the more recent approximately 20% of samples are placed into the test set, while at least one sample from the same family is retained in the training set. This split is designed to make the test samples later than the training samples whenever possible, while avoiding families that are completely unseen during training. The temporal split is first constructed over all 4383 APKs, yielding 3501 training APKs and 882 test APKs. Static parsing subsequently fails for 65 training APKs and 16 test APKs. After these 81 samples are excluded, 3436 training APKs and 866 test APKs remain for evaluation. Each of the 118 test families has at least one training reference. Accordingly, the protocol evaluates closed-set temporal attribution: Accuracy and Macro F1 measure attribution of later APK variants to known families rather than recognition of a family absent from training.

Model and parameter selection use an internal split of the 3436 parsed training APKs into 2757 subtrain APKs and 679 validation APKs. The text-model configuration and learned-fusion regularization are determined using only these subsets. Once these choices are fixed, the Text Analyzer and learned-fusion baselines are refitted on all 3436 training APKs and evaluated once on the 866 test APKs.

Labeled training APKs then serve as references and test APKs as queries. Three controls protect this protocol from leakage. First, split membership is assigned using APK SHA-256 identifiers, so one APK cannot appear as both a training reference and a test query. Second, model settings and training-derived statistics are fixed without using test APKs. Third, test-family labels are unavailable during candidate generation, similarity computation, neighbor retrieval, and family voting. They are used only after predictions have been produced to calculate Accuracy, Macro F1, and Top-3 Accuracy. The package-name and signing-certificate overlaps discussed in Section 6 remain an explicit limitation rather than being treated as duplicate APKs.

4.2. Compared methods and sensitivity analyses

This subsection introduces the comparison and sensitivity experiments in the order in which they are used. We first define the primary comparison using the majority-family and single-source baselines, followed by two supervised family classifiers. We then examine alternative fusion rules, repeat the ablation under each fusion rule, and test class-frequency correction in neighborhood voting. Finally, we vary the remaining family-attribution and graph-construction settings.

Primary and Single-Source Comparisons. To establish a reference for the complete framework and measure what each evidence source can achieve alone, we compare EVIDENCEMALGRAPH with a majority-family baseline and five single-source variants: Certificate-only, Package-only, Text-only, Infrastructure-only, and Behavior-only. The majority-family baseline provides a simple reference point, while the single-source variants isolate the attribution capability of each evidence source. The complete EVIDENCEMALGRAPH combines all five sources.

Supervised Family-Classification Baselines. To determine whether EVIDENCEMALGRAPH remains competitive beyond its own variants, we compare it with two supervised family classifiers under the same split. Following the widely adopted Drebin design (Arp et al., 2014), we implement a multiclass linear SVM that encodes permissions, components, API calls, URLs, and domains as sparse static features and learns from family labels rather than binary malware/benign labels. The DeepAMD-style static baseline (Imtiaz et al., 2021) applies a ReLU multilayer perceptron to permission and API-call features after training-only di-

mensionality reduction. For each baseline, the validation set is used to choose the best-performing model setting. The selected model is then trained again on all 3436 outer-training APKs before testing. These baselines are implemented under the same experimental protocol to enable a controlled comparison, rather than as exact reproductions of the original implementations.

Fusion Strategies and Training. To test whether the attribution result depends on how the five similarities are combined, we compare four fusion rules using the same candidate APK pairs and neighborhood-voting procedure. Uniform fusion gives the five sources equal weights. Knowledge-guided fusion uses the fixed weights in Eq. (4). Learned linear fusion estimates the five weights from training data. Learned interaction fusion also includes the ten possible two-source products. For example, package similarity multiplied by text similarity forms a separate term that is large only when both similarities are high.

The learned strategies also test whether differences among the five score scales affect fusion. They receive the five scores separately and learn how much each source should influence the choice of the closest labeled training APKs. For example, if one source gives high similarity scores even to different-family APKs, training can reduce its influence.

The learned methods compare a query APK q with a same-family reference a^+ and a different-family reference a^- . Here, $F_w(q, a)$ is their fused similarity under the learned weights w . Training seeks $F_w(q, a^+) > F_w(q, a^-)$, so the same-family APK is ranked first. The score is

$$F_w(q, a) = w^\top \phi(r(q, a)), \quad w_k \geq 0, \quad \sum_k w_k = 1, \quad (6)$$

The vector $r(q, a)$ contains the five evidence similarities. Linear fusion uses these five values directly, whereas interaction fusion appends their ten pairwise products through ϕ . The non-negative weights w sum to one and are learned by minimizing

$$\mathcal{L}(w) = \sum_{(q, a^+, a^-) \in D_{\text{rank}}} \log(1 + \exp[-(F_w(q, a^+) - F_w(q, a^-))]) + \lambda \|w - u\|_2^2, \quad (7)$$

In Eq. (7), the first term penalizes a different-family reference that scores above a same-family reference. The second keeps w near the equal-weight vector u , and λ controls this constraint. We test $\lambda \in \{0, 0.01, 0.1, 1\}$, covering no, weak, moderate, and strong constraints on a logarithmic scale. Validation Macro F1 selects $\lambda = 1$ for both learned methods. To limit family imbalance, each family contributes at most 512 comparisons. Weight learning uses 2757 subtrain APKs, parameter selection uses 679 validation APKs, and the selected models are refitted on their combined 3436 APKs. Test labels are used only for final evaluation.

Cross-Fusion Ablation Protocol. To determine whether the ablation findings are specific to the knowledge-guided weights, we repeat every source-removal experiment under all four fusion strategies. Knowledge-guided and uniform fusion renormalize the weights of the four remaining sources. Learned linear fusion is retrained without the removed source. Learned interaction fusion also excludes every interaction that contains that source before retraining. Candidate APK pairs, neighborhood voting, and data splits remain unchanged. Model settings are selected on validation data, and external-test labels are used only for final metrics.

Neighbor-Balance Comparison. To determine whether training-family size should alter the neighborhood vote, we compare four levels of frequency correction. This controlled comparison temporarily replaces the vote in Eq. (5) with $\sum S(q, r)/f(y)^\alpha$, while keeping candidate retrieval, similarity weights, neighbor-selection thresholds, and the adaptive one/three-neighbor rule fixed. We compare $\alpha \in \{0, 0.25, 0.5, 1\}$. The value $\alpha = 0$ applies no frequency correction, 0.25 is the original fourth-root correction, 0.5 applies a stronger square-root correction, and 1 is full inverse-frequency class-balanced kNN. The exponent is selected by validation Macro F1 and then Accuracy. External-test labels do not affect this selection.

Family-Attribution Parameter Sensitivity. To check whether family predictions depend on the remaining manually set neighborhood parameters, we vary one setting at a time. The maximum number of voting neighbors is set to 1, 3, or 5. The score gap for accepting the first neighbor alone is set to 0.01, 0.02, or 0.03. The high-similarity cutoff is set to 0.45, 0.50, or 0.55. Candidate retrieval and all other settings remain fixed. The main rule is retained according to the internal validation results, while external-test labels are used only to report the final sensitivity range.

Graph-Construction Sensitivity. To check whether the Campaign candidates depend on the fixed graph threshold, we compare fixed similarity cutoffs from 0.40 to 0.50 in steps of 0.025 with two adaptive alternatives. The first uses Otsu's method to select a cutoff from the candidate-pair score distribution by finding the strongest separation between its lower- and higher-similarity regions. The second constructs symmetric top- k graphs for $k \in \{1, 3, 5\}$, where an edge is retained when either APK ranks the other among its k most similar candidates. Family labels are not used to select edges or detect communities. They are used only after graph construction to calculate the evaluation metrics.

4.3. Evaluation metrics and bootstrap analysis

This subsection first defines the metrics for the four research questions. It then describes bootstrap resampling used to quantify uncertainty in the primary external-test family-attribution results.

Evaluation Metrics. Table 1 summarizes the evaluation metrics used in this work. For RQ1 and RQ2, classification performance is mainly reported using Accuracy, Macro F1, Top-3 Accuracy, and prediction coverage. Macro F1 is the primary metric because the number of samples varies greatly across families in MaIRadar. If only Accuracy is considered, large families may have a stronger influence on the result. Macro F1 first computes the performance for each family separately and then takes the average, making it more suitable for reflecting the method's ability to recognize small families. RQ2 conducts ablation experiments by removing one evidence source at a time. RQ3 evaluates the quality of the similarity graph and Campaign candidates. RQ4 evaluates whether the generated reports are complete, evidence-grounded, and reproducible.

Bootstrap Uncertainty Analysis. A single external test split yields one Accuracy value and one Macro F1 value, but these point estimates do not show how strongly the results depend on the particular APKs included in the test set. We use bootstrap simulation to examine this dependence. The 866 test APKs are repeatedly sampled with replacement within each of the 118 families. An APK may therefore appear more than once in a resampled set, while another APK from the same family may be absent. Each family still contributes the same number of APKs as it does in the original test set, so the long-tailed class distribution remains unchanged. Each resampled set represents another possible composition of test APKs drawn from the families in the current split.

For every resampled set, we recalculate Accuracy and Macro F1 for EVIDENCEMALGRAPH and Text-only from their fixed predictions. Both methods are evaluated on exactly the same resampled APKs. Their paired difference therefore reflects a change in sample composition rather than a difference in test data. We repeat this process 10,000 times with a random seed of 42 and use the 2.5th and 97.5th percentiles as the 95% confidence interval. If the interval for the paired improvement remains above zero, the fusion gain is unlikely to depend on a small group of test APKs. The procedure does not retrain or tune either method. Its confidence intervals describe sampling variation within the current test data, not generalization to other datasets or collection periods.

5. Results

This section answers the four research questions and presents the supporting comparison and robustness analyses. The RQ1 results in-

Table 1
Definitions of evaluation metrics used in this work.

Item	Definition	Used for
Accuracy	Proportion of test APKs whose predicted family is correct.	RQ1, RQ2
Macro F1	Average F1 across families, used as the primary metric for long-tailed family distributions.	RQ1, RQ2
Top-3 Accuracy	Proportion of test APKs whose true family appears in the top three predictions.	RQ1
Prediction Coverage	Proportion of test APKs for which a method can produce a prediction.	RQ1
Ablation Drop	Performance decrease after removing one evidence source from the full model under the same fusion strategy.	RQ2
Neighbor Coverage	Proportion of test APKs that can find at least one training neighbor.	RQ3
Same-family Edge Ratio	Proportion of retained or strong graph edges connecting APKs from the same family.	RQ3
Cross-family Candidate	Sample or community with different family labels but strong shared static evidence.	RQ3
Multi-sample Campaign Coverage	Proportion of APKs assigned to Campaign candidates containing at least two samples.	RQ3
Purity	Degree to which each discovered community is dominated by one family label.	RQ3
ARI	Agreement between discovered communities and family labels after correcting for chance.	RQ3
NMI	Amount of family-label information preserved by the community partition.	RQ3
Report Verification Metrics	Machine-checkable report constraints, including schema, source, citation, evidence, and Markdown checks.	RQ4

clude neighbor-balance, supervised-baseline, bootstrap, fusion, and parameter-sensitivity comparisons, followed by evidence-source ablation for RQ2, graph-based Campaign discovery for RQ3, and evidence-grounded report generation for RQ4.

RQ1. How well do individual evidence sources and the complete EvidenceMalGraph framework attribute external-test APKs to families represented in the training set?

Neighbor-Balance Comparison. Table 2 isolates the frequency correction in the voting rule. Removing the correction gives the highest validation Accuracy and Macro F1. The full inverse-frequency class-balanced variant is lower by 0.15 Accuracy points and 0.73 Macro F1 points on validation. The same ordering appears on the external test set, where full class balancing is lower by 0.46 Accuracy points and 1.21 Macro F1 points. We therefore use the uncorrected similarity vote in Eq. (5). In this adaptive neighborhood of at most three references, stronger global-frequency correction can discount a close reference simply because its family has more training samples.

Fusion Strategy Comparison. Table 3 reports their validation and external-test results. On the internal validation set, the knowledge-guided fusion obtains 93.67% Accuracy and 80.42% Macro F1, the highest values among the four strategies. The interaction and linear rankers obtain 79.97% and 79.29% Macro F1, respectively, while uniform fusion obtains 77.90%. We therefore retained the knowledge-guided configuration before evaluating the external test set.

On the external test set, learned linear rank fusion reaches 90.99% Accuracy and 73.85% Macro F1, exceeding the main configuration by 0.23 and 0.64 percentage points. This gain does not appear on the validation set, where the linear ranker is lower by 0.15 Accuracy points and 1.13 Macro F1 points. Because the external test labels were unavailable when the main strategy was retained, we report the test gain as a comparison rather than using it to replace the main configuration after seeing the test outcome. Across all four fusion strategies, external-test Accuracy varies from 90.76% to 90.99% and Macro F1 from 73.21% to 73.85%. Neither learned alternative shows a consistent advantage across validation and test.

Family Attribution Performance Analysis. Based on the validation comparisons above, we select $\alpha = 0$ and knowledge-guided fusion for the main model before examining the external test results. Fig. 2 presents the family attribution performance of this fixed configuration on the external test set. In this setting, each test APK is compared only with APKs in the training set that have known family labels. The test labels are used only for final evaluation and do not participate in the prediction process. The figure reports the majority-family baseline, the five single-source variants, and the complete EVIDENCEMALGRAPH, allowing us to compare each evidence source used alone with the overall performance after multi-source fusion. *On the external test set, EvidenceMalGraph achieves an Accuracy of 90.76% and a Macro F1 of 73.21%, outper-*

Table 2
Effect of class-frequency correction in neighborhood voting.

Voting strategy	Validation		External test	
	Accuracy	Macro F1	Accuracy	Macro F1
No frequency correction (main)	93.67	80.42	90.76	73.21
Fourth-root correction ($\alpha = 0.25$)	93.52	79.81	90.76	72.93
Square-root correction ($\alpha = 0.5$)	93.52	79.69	90.42	72.35
Full class-balanced kNN ($\alpha = 1$)	93.52	79.69	90.30	72.00

Note: Only the exponent applied to training-family frequency changes, while the main strategy is selected on the internal validation set.

Table 3
Comparison of fixed and learned fusion strategies.

Fusion strategy	Validation		External test	
	Accuracy	Macro F1	Accuracy	Macro F1
Knowledge-guided fusion (main)	93.67	80.42	90.76	73.21
Uniform fusion	93.08	77.90	90.76	73.21
Learned linear rank fusion	93.52	79.29	90.99	73.85
Learned interaction rank fusion	93.52	79.97	90.88	73.85

forming all single-source variants. Compared with the strongest single source, Text-only, the complete model improves Accuracy by 3.58 percentage points and Macro F1 by 4.00 percentage points. The simultaneous improvement in both metrics is important. The increase in Accuracy indicates that the overall predictions are more accurate, while the increase in Macro F1 shows that the gain does not only come from a few families with the largest numbers of samples, but also extends to some smaller families.

Among the single-source variants, Text-only performs the best, with an Accuracy of 87.18% and a Macro F1 of 69.21%. This is consistent with the characteristics of Android malware. Many samples reuse deceptive texts, fake update prompts, phishing vocabulary, or similar business templates. Behavior-only ranks second, with an Accuracy of 84.99% and a Macro F1 of 63.28%. It captures another type of signal, namely the combination of permissions, components, and sensitive capabilities. In simple terms, textual evidence is closer to how a sample deceives users, while behavioral evidence is closer to what a sample is prepared to do. Package-only, Infrastructure-only, and Certificate-only perform weaker when used alone, but this does not mean that these types of evidence are not valuable. Package-only achieves an Accuracy of 80.37%, indicating that package structure is still useful for families with clear code-template reuse. However, its Macro F1 is only 55.63%, suggesting that this clue is not stable enough for long-tail families. Infrastructure-only has a coverage of 96.42% because some APKs do not expose usable domains, URLs, or network endpoints. Certificate-only has a coverage of only 46.77%. This means that certificate matching is often a strong clue

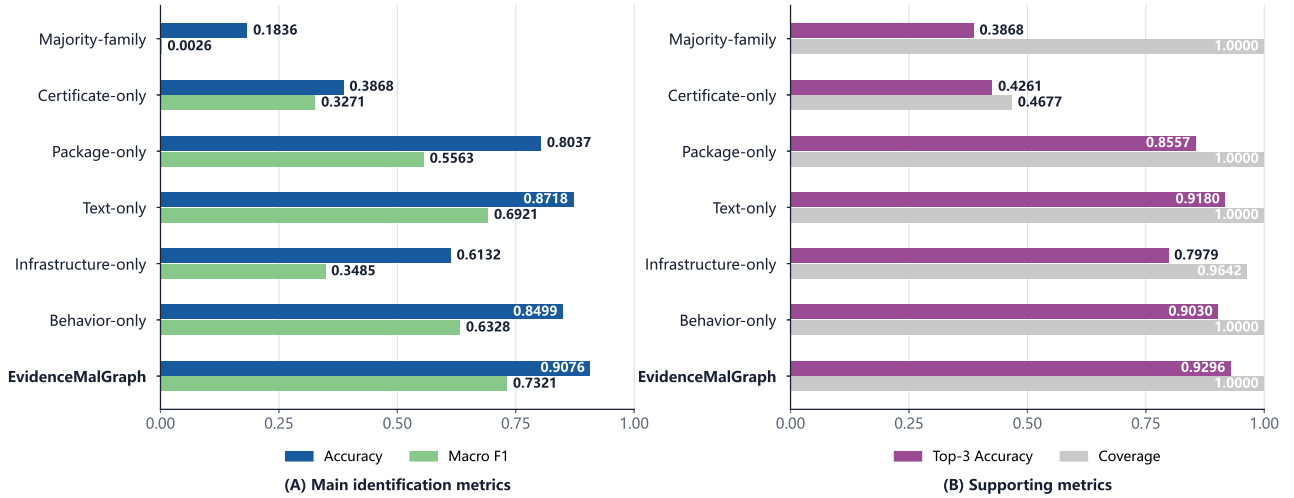


Fig. 2. External-test performance of individual evidence sources and EVIDENCEMALGRAPH.

once it appears, but it does not cover enough test samples to support family attribution on its own.

The advantage of the complete model comes from the mutual correction among these types of evidence. Textual similarity may arise simply because multiple families use the same advertising templates or deceptive expressions. Behavioral similarity may also come from common malicious capabilities rather than from the same family. Package names and certificates are also easily affected by repackaging. When only one type of evidence is considered, these similarities can be overemphasized. After fusion, the system tends to favor candidate families that are supported by multiple types of evidence at the same time. Therefore, even though Text-only is already strong, Macro F1 continues to improve in the complete model.

Top-3 Accuracy reaches 92.96%, which is 2.20 percentage points higher than Top-1 Accuracy. This gap indicates that, for some samples, although the final predicted label is incorrect, the true family still appears among the top three candidates. For scenarios that require manual review, such candidate lists are useful. Analysts can first examine the top-ranked families instead of searching again from all families. However, this work still treats Top-1 Accuracy and Macro F1 as the main conclusions, because an automatic attribution system ultimately needs to output a definite label.

The leave-one-out evaluation on the full dataset achieves an Accuracy of 95.25% and a Macro F1 of 82.27% on 4249 attributable samples, which are higher than the external test results. However, the leave-one-out results should not be directly interpreted as generalization performance. In leave-one-out evaluation, the reference samples and query samples come from the same data collection, which creates more opportunities for overlaps in package names, certificates, text templates, and behavior patterns. Therefore, it is more suitable for observing evidence consistency within the dataset. The external holdout setting is closer to practical use, where existing samples are used to attribute newly observed samples. Therefore, the assessment of each evidence source in RQ1 is still based on the external test results shown in Fig. 2.

Bootstrap Uncertainty Analysis. Fig. 3 summarizes both the uncertainty of the absolute performance and the improvement of EVIDENCEMALGRAPH over Text-only, based on 10,000 bootstrap resamples of the 866 external-test APKs. Fig. 3(a) compares the two methods directly. EVIDENCEMALGRAPH obtains 90.76% Accuracy with a 95% confidence interval of 89.49%–92.03%, and 73.21% Macro F1 with an interval of 71.08%–75.03%. Text-only obtains 87.18% Accuracy with an interval of 85.57%–88.80%, and 69.21% Macro F1 with an interval of 66.98%–71.54%. Fig. 3(b) shows the paired improve-

Table 4

External-test comparison with supervised family-classification baselines.

Method	Accuracy	Macro F1	Top-3 Accuracy
EvidenceMalGraph (main)	90.76	73.21	92.96
Drebin-inspired multiclass SVM	88.45	69.91	90.18
DeepAMD-style static MLP	87.64	66.13	92.96

Table 5

Sensitivity of family attribution to neighborhood settings.

Neighborhood rule	Validation		External test	
	Accuracy	Macro F1	Accuracy	Macro F1
Main: $k_{\max} = 3$, gap 0.02, cutoff 0.50	93.67	80.42	90.76	73.21
Maximum neighbors $k_{\max} = 1$	93.52	80.27	90.65	73.06
Maximum neighbors $k_{\max} = 5$	93.37	79.14	91.11	73.30
Dominance gap 0.01	93.52	80.40	90.53	72.78
Dominance gap 0.03	93.52	79.49	90.88	73.47
High-similarity cutoff 0.45	93.67	80.30	90.76	73.27
High-similarity cutoff 0.55	93.67	80.42	90.53	72.88

Note: Each alternative changes one setting while retaining the other two main values, with selection based only on the validation set.

ment calculated by evaluating both methods on exactly the same resampled APKs. EVIDENCEMALGRAPH improves Accuracy by 3.58 percentage points, with a 95% confidence interval of 2.42–4.85 points, and improves Macro F1 by 4.00 points, with an interval of 2.41–5.14 points. Both intervals remain above zero, and EVIDENCEMALGRAPH outperforms Text-only in all 10,000 resamples. The fusion gain is therefore stable when the APK composition of the current test set is varied through resampling.

Learning-Based Baseline Comparison. Table 4 reports the external-test comparison with the two supervised family-classification baselines. EVIDENCEMALGRAPH obtains the highest Accuracy and Macro F1. Compared with the Drebin-inspired SVM (Arp et al., 2014), it improves Accuracy by 2.31 percentage points and Macro F1 by 3.30 points. Compared with DeepAMD-style static MLP (Imtiaz et al., 2021), EVIDENCEMALGRAPH places the correct family first for 786 rather than 759 test APKs, a difference of 27 samples. Both methods include the correct family among their first three candidates for 805 APKs. Their Top-3 coverage is therefore identical, while EVIDENCEMALGRAPH is more reliable in placing the correct family first and performs better across families.

Family-Attribution Parameter Sensitivity. Table 5 reports the results obtained by changing one neighborhood setting at a time. The main rule reaches or ties the highest validation Accuracy and Macro F1. Across all

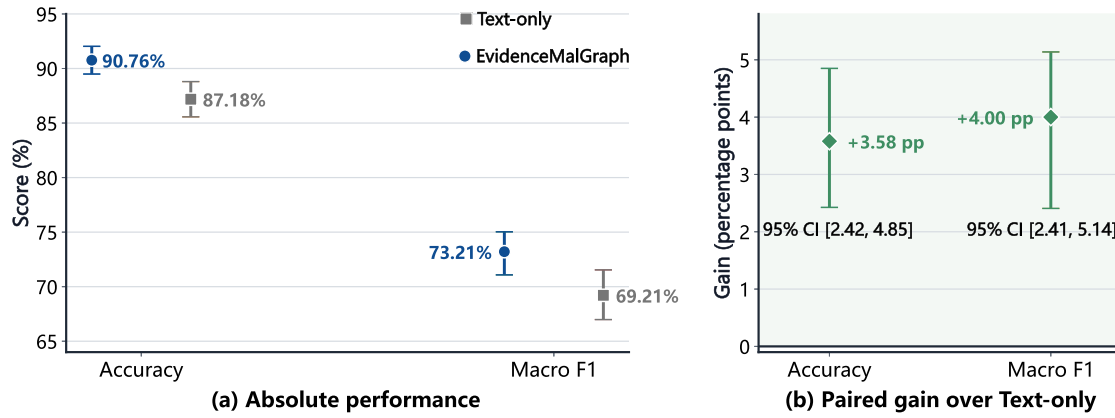


Fig. 3. External-test performance and paired gain with 95% bootstrap confidence intervals.

seven settings, validation Accuracy ranges from 93.37% to 93.67%, and Macro F1 ranges from 79.14% to 80.42%. The corresponding external-test ranges are 90.53–91.11% and 72.78–73.47%. Some alternatives are slightly higher on the external test set but lower on validation. We therefore retain the main rule selected without using external-test labels.

Answer to RQ1. Text-only and Behavior-only are the strongest single-source variants, but EVIDENCEMALGRAPH achieves 90.76% Accuracy and 73.21% Macro F1 through five-source evidence fusion. On the internal validation set, the knowledge-guided weights achieve the highest Accuracy and Macro F1. It also exceeds the Drebin-inspired SVM and DeepAMD-style static MLP on both primary external-test metrics.

RQ2. How does family attribution performance change when each evidence source is removed, and which sources contribute most to the fused result?

As shown in Fig. 4, the main ablation uses the selected knowledge-guided fusion, and the most evident result in the ablation study comes from the textual evidence. **After removing textual evidence, the overall attribution accuracy decreases from 90.76% to 87.76%, and Macro F1 decreases from 73.21% to 70.30%.** This result is consistent with the single-source experiment in the previous section. Many Android malware samples retain similar deceptive expressions, fake update prompts, login texts, gambling vocabulary, or phishing keywords. These texts are not irrelevant noise, but direct clues for family attribution.

The effect of behavioral evidence is not as large as that of the textual evidence, but the performance drop is also stable. After removing it, Accuracy decreases by 1.50 percentage points, and Macro F1 decreases by 2.74 percentage points. It complements textual evidence with another type of information, including what permissions a sample requests, which components it registers, and whether sensitive capabilities such as SMS abuse, device administration, overlay attacks, or persistence appear. Even if an APK changes its interface text or resource files, this type of evidence can still link it to related families as long as its behavior combination does not change completely.

The change caused by removing the infrastructure evidence is relatively small. Accuracy decreases by only 0.23 percentage points, and Macro F1 decreases by 0.21 percentage points. This is not surprising. Network infrastructure clues are often useful, but they are not available for every sample. Some APKs do not expose clear domains or URLs, and some families may change server-side addresses across different versions. Therefore, in the current dataset, infrastructure evidence acts more like a supplementary signal. It can strengthen attribution when

clear reuse appears, but it cannot determine the result when it is missing.

The more nuanced cases are package and certificate evidence. Removing either source leaves Accuracy unchanged at 90.76%. Macro F1 changes only slightly, reaching 73.33% without package evidence and 73.27% without certificate evidence. These differences are too small to support a positive classification contribution in the current external test. One possible reason is the presence of public test certificates, batch re-signing, package-name reuse, repackaging templates, and exact package-name or certificate overlaps between the training and test sets. Here, it is necessary to distinguish classification contribution from analytical value. Certificates and package structures remain useful for explaining why samples are similar and for tracing evidence-based sample relationships, even when removing them has almost no effect on family-classification metrics

Cross-Fusion Ablation Robustness. Because the exact drop can depend on the fusion rule, Table 6 repeats the five ablations under knowledge-guided, uniform, learned linear, and learned interaction fusion. Text remains the largest contributor under all four strategies. Its removal reduces external-test Accuracy by 2.19–3.00 percentage points and Macro F1 by 2.91–3.65 points. Behavior is also consistently helpful, with drops of 1.50–2.08 and 1.56–2.74 points, respectively. Infrastructure has a smaller positive effect, while certificate and package evidence do not show a stable classification gain. The validation results show the same Text–Behavior ordering. Thus, the exact drop values change with the fusion strategy, but the main contribution pattern is not specific to the preset weights.

Answer to RQ2. Across all four fusion strategies, textual evidence contributes the largest improvement, behavioral evidence provides consistent complementary gains, and infrastructure evidence adds a smaller positive signal. Package and certificate evidence do not provide a stable classification gain, although they remain useful for explaining sample relationships.

RQ3. How well do the graph communities group APKs with consistent static evidence and family labels, and can they surface cross-family relationships for further investigation?

RQ3 evaluates graph coverage and the extent to which static-evidence connections and detected communities are consistent with existing family labels. Since MalRadar provides no Campaign ground truth, these measurements cannot establish that a community represents a confirmed real-world Campaign. In the training-reference and test-query graph, every test sample can find at least one training neighbor, with a neighbor coverage of 100%. Among them, **93.65% of test samples have at least one strong connection with a similarity score of 0.45 or**

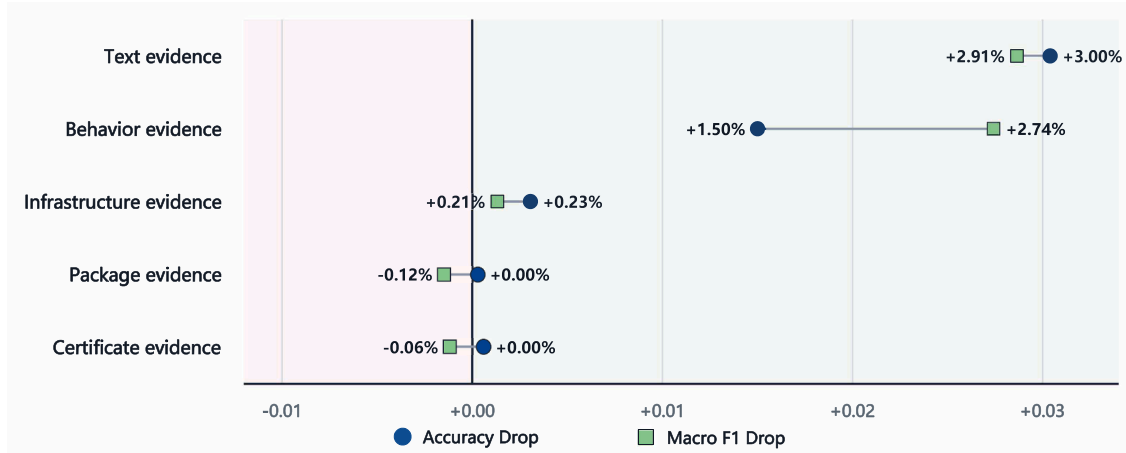


Fig. 4. Performance drop after removing each evidence source under knowledge-guided fusion.

Table 6

External-test ablation drops across fusion strategies.

Fusion strategy	Certificate	Package	Text	Infrastructure	Behavior
Knowledge-guided	+0.00/ - 0.06	+0.00/ - 0.12	+ 3.00/ + 2.91	+0.23/ + 0.21	+ 1.50/ + 2.74
Uniform	+0.00/ - 0.16	-0.81/ - 1.97	+ 2.19/ + 3.03	+0.58/ + 0.30	+1.85/ + 1.69
Learned linear	+0.00/ + 0.06	-0.12/ - 0.99	+2.43/ + 3.65	+0.23/ + 0.86	+ 2.08/ + 1.56
Learned interaction	-0.46/ - 1.23	-0.35/ - 0.88	+2.31/ + 3.59	+0.58/ + 1.21	+1.96/ + 2.73

Note: Each cell reports Accuracy drop / Macro F1 drop in percentage points. Positive values mean that removing the source reduces performance.

higher. The proportion of nearest neighbors belonging to the same family is 90.65%, and the proportion of strong connections linking samples from the same family is 97.87%. In other words, most strong edges retained by the system are not arbitrary connections. They usually connect samples with consistent family labels.

The more interesting part lies in the cross-family results. The system identifies 81 test samples with different labels but highly similar evidence, and each candidate is supported by at least two types of evidence. Such results cannot be directly interpreted as evidence that two families belong to the same attacker, but they are suitable as entry points for further investigation. They may result from shared malicious components, family evolution, or infrastructure reuse. They may also be caused by public SDKs, similar fraud templates, repackaging processes, or label noise in the dataset. Therefore, this work refers to them as Campaign candidates rather than confirmed real-world attack operations. **On all 4302 successfully analyzed samples, community detection identifies 244 campaign candidates, including 120 multi-sample candidates and 124 singleton candidates.** The multi-sample coverage is 97.12%, indicating that the vast majority of APKs can be assigned to investigable sample groups rather than remaining isolated in the graph. The edge family homogeneity is 97.68%, meaning that approximately 97.68% of the retained similarity edges connect samples from the same family. This value is high, suggesting that the local connections in the graph are relatively clean.

The community-level metrics further quantify agreement with existing family labels rather than agreement with real Campaign ground truth. Purity reaches 87.63%, indicating that most communities contain a dominant family rather than a random mixture of many families. ARI reaches 75.56%, showing a high level of agreement between the community partition and the family labels, and this agreement is not simply produced by random grouping. NMI reaches 87.71%, indicating that the community partition preserves substantial family-label information. In other words, the communities identified by EVIDENCE-MALGRAPH are generally consistent with existing family labels. Samples from the same family are usually grouped together. However, communities are not completely equivalent to family labels. The system also groups a small number of APKs with different labels but highly similar

Table 7

Sensitivity of the APK similarity graph to edge-selection rules.

Graph rule	Communities	Coverage	Edge homogeneity	ARI	NMI
Fixed cutoff 0.400	148	98.19	93.95	63.29	80.81
Fixed cutoff 0.425	193	97.79	96.67	72.24	84.80
Fixed cutoff 0.450 (main)	244	97.12	97.68	75.56	87.71
Fixed cutoff 0.475	317	96.05	98.20	76.44	88.74
Fixed cutoff 0.500	375	95.03	98.36	77.11	89.23
Otsu cutoff 0.443	233	97.26	97.52	75.62	87.43
Symmetric top-1	1034	100.00	93.51	4.32	66.82
Symmetric top-3	224	100.00	90.85	17.24	71.80
Symmetric top-5	125	100.00	88.70	22.69	72.61

Note: Coverage is the proportion of APKs in communities containing at least two samples. Otsu uses only candidate-pair scores. Family labels are used only to compute homogeneity, ARI, and NMI.

static evidence into the same community. These cross-family communities are exactly the cases that require focused manual investigation in subsequent analysis.

Graph-Construction Sensitivity. Table 7 shows the trade-off created by the graph rule. Lower fixed cutoffs retain more APKs in multi-sample communities but also admit more cross-family edges. Higher cutoffs improve edge homogeneity and label agreement but split the graph into more communities and reduce coverage. The main cutoff of 0.45 gives 97.12% multi-sample coverage, 97.68% edge homogeneity, and 244 communities.

The label-free Otsu rule selects 0.443, close to the main cutoff of 0.45, and produces similar coverage, edge homogeneity, ARI, and NMI. This independent estimate supports 0.45 as a reasonable balance for the observed score distribution. Top- k graphs connect every APK, but they must retain a prescribed number of neighbors even when the available similarities are weak. Top-1 consequently fragments the graph into 1034 communities, while top-3 and top-5 reduce edge homogeneity to 90.85% and 88.70%. The fixed cutoff is therefore retained because it requires every edge to meet a common evidence-strength criterion rather than forcing a connection for each APK.

To further illustrate the analytical value of Campaign candidates, we select Campaign 10 as a representative case based on its cross-family status, number of families, evidence-category diversity, and number of APKs. This Campaign contains 135 APKs and involves 25 families, with `solid` being the largest family with 61 APKs and `hiddad` being the second largest with 44 APKs. All 135 samples show evidence of command execution, file collection, foreground persistence, and device-state collection. In addition, 134 samples contain the fake update installation theme, and 132 samples show evidence of app discovery, location tracking, and overlay attacks. The significance of this community is not merely that it groups similar samples together. More importantly, it turns cross-family shared evidence into a testable analytical hypothesis. These samples may reuse the same malicious components, repackaging chains, or fraud templates. They may also reflect granularity differences in existing family labels. In other words, Campaign 10 provides an entry point for extending the analysis from individual APK similarity to family-boundary auditing and evidence-reuse pattern analysis.

Answer to RQ3. EVIDENCEMALGRAPH produces graph communities with broad coverage and high consistency with existing family labels, achieving 100% neighbor coverage, 97.87% same-family consistency among strong edges, and 97.12% multi-sample Campaign coverage. It also surfaces cross-family Campaign candidates for further investigation.

RQ4. Can the analysis reports automatically generated for Campaigns provide clear evidence, complete content, and reproducible verification?

Each Campaign generates one structured JSON file and one Markdown report. Reports for multi-sample Campaigns are written by `deepseek-v4-flash`, but the input contains only privacy-preserving aggregated evidence, such as the number of APKs, family distribution, behavior categories, and text categories. Singleton Campaigns do not invoke `deepseek-v4-flash` and are generated directly using a deterministic template. In total, 120 multi-sample reports are generated by `deepseek-v4-flash`, and 124 singleton reports are generated by the template, covering all 244 Campaign candidates. The report evaluation does not rely on subjective manual scoring. Instead, it checks whether each report can be verified by machines. **All reports achieve 100.00% on Schema Compliance, Source Reproducibility, Evidence Catalog Consistency, Claim Citation Coverage, Citation Validity, Evidence Category Coverage, and Markdown Completeness.** This means that the report fields are complete, the source summaries can be recomputed from the current Campaign evidence, the cited evidence entries are consistent with the local evidence catalog, each claim contains a valid `evidence_id`, and the Markdown report includes the necessary conclusions, evidence, and limitations.

The most important implication of this result is that the reports do not generate conclusions from scratch. Each claim must cite evidence actually extracted by the system, and the evaluator can use the same input to recheck whether these citations are valid. In other words, the system restricts `deepseek-v4-flash` to the role of organizing language based on existing evidence, rather than allowing it to determine sample families, add external threat intelligence, or freely infer attacker identities. The role of template reports is also reflected here. For Campaigns containing only one APK, the available evidence is limited. If a LLM is still invoked in such cases, it is more likely to expand limited evidence into over-interpretation. Using a deterministic template keeps singleton reports concise and stable, while also avoiding unnecessary token consumption. Multi-sample Campaigns, in contrast, contain richer aggregated evidence and are therefore more suitable for `deepseek-v4-flash` to generate more readable summaries.

Answer to RQ4. The generated Campaign reports are evidence-grounded, complete, and reproducible, with all reports achieving 100% on schema compliance, source reproducibility, citation validity, evidence coverage, and Markdown completeness.

6. Threats to validity

This section organizes the threats into external, internal, and construct validity. External validity concerns the single-corpus, known-family temporal evaluation and the limits of bootstrap resampling for cross-dataset generalization. Internal validity concerns static-analysis coverage, leakage controls, validation-based fusion and voting choices, and differences between our learning-based baselines and the original systems. Construct validity concerns family-label reliability, the absence of real Campaign ground truth, and the scope of automated report-quality metrics.

6.1. External validity

The experimental subjects in this work are malicious APKs from `MalRadar`. Benign apps are not included, and the experiments are not repeated on other Android malware corpora. Therefore, the results of this work only support family attribution, nearest-neighbor retrieval, and Campaign candidate analysis among malicious samples. They should not be directly interpreted as malware/benign detection capability. When using a different dataset, family naming rules, sample collection periods, obfuscation levels, and repackaging ratios may all change, and the performance of EVIDENCEMALGRAPH needs to be revalidated. The experiments still use one main temporal split. The 10,000-iteration family-stratified bootstrap simulation shows how much the reported scores change when the APK composition of the current test set is resampled. The training/test assignment, the 118 test families, the collection period, and the data source remain fixed. The resulting confidence intervals therefore describe uncertainty within this test setting. They do not show how performance would change under a different split, collection period, or dataset. The split strategy can still affect the number of long-tail families and the degree of package-name or certificate reuse. Future work should therefore repeat the evaluation on additional collection periods and Android malware datasets.

All 118 test families are represented in the training set. This design evaluates temporal generalization to later APKs from known families, but it does not establish open-set generalization to families absent from training. A proper open-set evaluation would require complete families to be withheld, an unknown-family rejection rule selected without external-test labels, and metrics for both unknown-family detection and known-family attribution. The current framework does not include this rejection mechanism, so unseen-family recognition remains outside the claims of this study. It should be noted that the main split still contains 196 distinct `package_name` strings and 171 signing-certificate SHA-256 hashes that appear in both the training and test sets. The package-name overlap here does not mean that APK samples are duplicated. Each APK is still uniquely identified by its SHA-256 hash. The package name is only the app identifier declared in the Manifest, and it may be reused by multiple APKs in malware datasets due to version updates, repackaging, impersonation, or batch generation. This work retains and reports these overlaps because they reflect common phenomena in Android malware data, including package-name reuse and certificate reuse. At the same time, such overlaps may make the test task easier. Therefore, this work lists them as a threat to external validity.

6.2. Internal validity

Static analysis itself has blind spots. Encrypted strings, dynamically loaded code, reflection, native code, and conditionally triggered logic may all evade static extraction by `AndroGuard`. In the experiments, 81

APKs also failed to be parsed successfully. If these failed samples are concentrated in certain families or specific obfuscation styles, the final results may underestimate the impact of difficult samples. The main fusion weights express prior assumptions about evidence reliability rather than parameters estimated from the external test set. We compare them with uniform fusion and two supervised ranking-based alternatives, with all learned settings selected on the internal validation split. The learned methods do not show a consistent advantage across validation and test, but this comparison still relies on one internal split. A different validation partition or malware corpus may favor another fusion function. Repeated nested validation across additional datasets would provide a stronger assessment of weight stability.

The evaluation controls described in Section 4.1 address three sources of optimistic bias: duplicate APKs appearing in both splits, statistics learned from test samples, and test labels entering the prediction process. They therefore support the validity of the reported results for later APKs whose families are already represented in the training set. The controls do not establish open-set family recognition or cross-dataset generalization, and they do not remove the package-name and certificate reuse discussed above. These cases remain outside the claim supported by the present experiment.

The learning-based baselines use the same data split and evaluation protocol as EVIDENCEMALGRAPH, but they are not complete reproductions of the original implementations. We implement representative feature and classifier designs from the published methods using the static evidence available in this study. Differences in preprocessing, software, and training details may therefore affect their results. Accordingly, Table 4 shows how these representative designs perform under the present protocol rather than establishing a definitive ranking against the original systems.

The decision to omit frequency correction is based on one internal validation split. On the current external test set, this setting retains the highest Macro F1 and ties for the highest Accuracy. However, a dataset with a different distribution of family sizes may benefit from frequency correction, so this choice should be validated again when the dataset changes.

6.3. Construct validity

Family labels are derived from the `report_family` and `AVClass_family` fields in the `sample-info.csv` metadata file provided by MalRadar. Such external metadata may contain aliases, inconsistent granularity, and labeling errors (Sebastián & Caballero, 2020). Because no independent relabeling or manual adjudication is performed, the reported scores may change under a corrected or differently normalized family-label set. Therefore, Accuracy and Macro F1 measure how well EVIDENCEMALGRAPH fits the existing label system rather than providing an absolute judgment of true family identities. Campaign evaluation faces a similar issue. MalRadar does not provide Campaign ground truth such as attacker identities, infrastructure ownership, or propagation timelines. Therefore, Purity, ARI, NMI, and edge family homogeneity can only use family labels as proxies. These metrics can indicate whether graph communities are consistent with existing labels, but they cannot prove that a cross-family community is necessarily operated by the same attacker. For this reason, this work refers to these communities as Campaign candidates rather than confirmed real-world attack Campaigns.

The report-quality metrics cover structural and factual constraints. Schema Compliance, Citation Validity, and Source Reproducibility can verify whether report fields are complete, whether citations are valid, and whether sources can be recomputed, but they do not evaluate whether the language is concise or whether the threat reasoning reaches expert level. The service version, model updates, and output strategy of `deepseek-v4-flash` may also affect the natural-language content. The current system already stores structured responses and source sum-

maries, but future work should still record more precise model versions, invocation dates, and prompt hashes.

7. Related work

This section reviews prior studies related to EVIDENCEMALGRAPH from three perspectives: Android malware detection, malware clustering and Campaign analysis, and explainable malware reporting.

7.1. Malware detection

Android malware static analysis does not require executing APKs, making it suitable for large-scale extraction of Manifest information, DEX files, signing certificates, strings, permissions, components, and API calls. Drebin (Arp et al., 2014) is an early representative work that combines lightweight static features with explainable detection. It extracts features such as permissions, APIs, network addresses, and components directly on the device, and presents malicious patterns to users. DroidAPIMiner (Aafer et al., 2013) further mines API-level features, showing that sensitive API calls provide stable discriminative power for Android malware detection. DroidSIFT (Zhang et al., 2014) uses weighted contextual API dependency graphs to characterize program behavior, emphasizing that API calling context is more suitable than simple frequency features for handling repackaging and obfuscation. These works mainly focus on malware detection, but they also provide a reusable static-evidence foundation for subsequent family attribution.

Later work explored behavior abstraction, sequence models, and multi-view representations. MaMaDroid (Mariconti et al., 2017) abstracts API calls in static call graphs to the package or API-family level, and models call sequences using Markov chains to improve detection across time. MKLDroid (Narayanan et al., 2018) constructs dependency graphs from multiple program views and fuses different views through multiple-kernel learning, demonstrating the necessity of multi-source static evidence fusion. HinDroid (Hou et al., 2017) uses heterogeneous information networks to model relationships among apps, APIs, signatures, and other entities, showing that graph structures and meta-paths can provide contextual information beyond a single feature vector.

Several later studies evaluate malware family classification directly. MalDozer (Karbab et al., 2018) uses API method sequences for both malware detection and family classification. DeepAMD (Imtiaz et al., 2021) uses a deep neural network to identify malware families from static and dynamic features, while GDroid (Gao et al., 2021) formulates family classification as node classification on an app-API graph. More recently, Meta-MAMC (Li et al., 2024) uses meta-learning and task-sampling strategies to address imbalanced and sparsely represented families. Unlike Drebin, MaMaDroid, MKLDroid, and HinDroid, whose principal evaluations concern malware detection, these studies explicitly report family-level classification.

Although the studies above use different features and learning models, their evaluations mainly center on malware detection or family-classification outcomes. EVIDENCEMALGRAPH extends the analysis beyond the predicted label by retaining the five source-specific similarities after fusion and attaching them to graph edges. Analysts can therefore examine whether a family prediction or downstream Campaign relationship is supported by certificate, package, text, infrastructure, or behavior evidence.

7.2. Malware clustering and campaign analysis

Malware analysis needs not only to determine which family a sample belongs to, but also to group large numbers of samples, identify shared components, and discover cross-family relationships that deserve further investigation. Traditional desktop malware studies have already used clustering for sample triage and family discovery. Malheur (Rieck et al., 2008) uses dynamic behavioral features for malware clustering,

demonstrating the value of behavioral similarity in large-scale malware analysis. BitShred (Jang et al., 2011) supports large-scale malware similarity search and grouping through feature hashing, highlighting the importance of scalable approximate similarity for security operations. The behavioral clustering work by (Bayer et al., 2009) also shows that automatic clustering can help discover malware variants and new families.

The Android scenario introduces additional difficulties. Third-party libraries are widely used, repackaged samples are common, and global code similarity can easily connect unrelated samples. (Li et al., 2017) propose iteratively mining malicious payloads to filter the interference of third-party libraries, and validate the effectiveness of payload-level clustering on the Android Genome dataset. Euphony (Hurier et al., 2017) and AVClass (Sebasti'an et al., 2016) focus on another fundamental problem: antivirus labels are highly noisy, the same family may have multiple aliases, and different vendors may use inconsistent labeling granularity. AVClass2 (Sebasti'an & Caballero, 2020) further extracts multidimensional tags such as family, category, behavior, and file attributes from AV labels, providing an automated tool for large-scale sample retrieval and label maintenance. AVScan2Vec (Joyce et al., 2023) treats antivirus scan reports as lightweight semantic inputs and learns representations suitable for production-scale malware corpora, supporting classification, clustering, and nearest-neighbor retrieval.

The studies above end with sample groups, similarity-search results, or normalized antivirus labels. EVIDENCEMALGRAPH combines family attribution and community analysis while keeping the two outputs distinct. Family labels come from supervised neighbor voting, whereas graph communities are treated as Campaign candidates. Louvain operates on an evidence graph in which each retained edge stores the five similarity components. Analysts can therefore inspect whether a community is held together by certificate, package, text, infrastructure, or behavior evidence, including for cross-family links.

7.3. Explainable malware reporting

Security analysis results need to help analysts understand why the system makes a particular judgment. Drebin (Arp et al., 2014) has already treated explainability as an important goal of Android malware detection by presenting static malicious patterns to help users understand the basis of detection. General machine learning explanation methods also provide useful reference frameworks. LIME (Ribeiro et al., 2016) explains the predictions of arbitrary classifiers using local interpretable models. SHAP (Lundberg & Lee, 2017) unifies different feature-contribution explanation methods based on Shapley values. GNNExplainer (Ying et al., 2019) extracts key subgraphs and node features for graph neural networks, showing that explanations for graph models should consider both structure and features. These methods emphasize that explanations must be faithful to model evidence, rather than merely providing natural-language statements that sound plausible.

LLMs provide fluent expression capabilities for automatic report generation, but they also introduce the risks of hallucination and unverifiable citations. RAG (Lewis et al., 2020) combines retrieved external evidence with generative models in an attempt to improve factuality and traceability in knowledge-intensive tasks. SelfCheckGPT (Manakul et al., 2023) and Chain-of-Verification (Dhuliawala et al., 2024) study hallucination mitigation from the perspectives of black-box consistency checking and post-generation verification, respectively. ALCE (Gao et al., 2023) further proposes a benchmark for automatically evaluating the citation quality of LLMs, pointing out that even when models can generate citations, the generated content often lacks complete evidential support.

Unlike these approaches, EVIDENCEMALGRAPH limits the LLM to privacy-preserving Campaign-level evidence and requires every report claim to cite a valid entry in a local evidence catalog. Unsupported

claims are removed before the report is saved. The catalog is built from the same analyzer records that produce the source-specific similarities used for family attribution and Campaign graph construction. Family predictions, graph relations, and report statements can therefore be checked against a consistent set of APK-derived evidence. The LLM organizes supported findings into natural language, while structured evidence and automatic checks determine what the report is allowed to claim.

8. Conclusion

This work develops EVIDENCEMALGRAPH for Android malware family attribution and Campaign relationship analysis. The modular framework links family attribution, Campaign candidate discovery, and report generation through a shared, traceable evidence pipeline. Five specialized analyzers retain source-specific observations and similarity scores for certificates, package structure, text, infrastructure, and behavior rather than hiding them in a single representation. The resulting evidence supports family predictions, explains the graph edges used to form Campaign candidates, and constrains the claims included in Campaign reports. This design preserves an evidence path from an analyzed APK to its predicted family, its relationships with other samples, and the corresponding report statements.

On the external test set, EVIDENCEMALGRAPH achieves 90.76% Accuracy and 73.21% Macro F1. It exceeds Text-only by 3.58 and 4.00 percentage points and outperforms the Drebin-inspired SVM and DeepAMD-style MLP on both primary metrics. Across 10,000 bootstrap resamples, the paired-gain intervals over Text-only remain above zero for both metrics. Validation selects knowledge-guided fusion and the uncorrected neighborhood vote with $\alpha = 0$. Learned fusion provides no consistent gain across validation and test, while the neighborhood-parameter sensitivity analysis keeps external Accuracy between 90.53% and 91.11% and Macro F1 between 72.78% and 73.47%. Cross-fusion ablations continue to identify textual and behavioral evidence as the largest contributors.

Beyond family attribution, EVIDENCEMALGRAPH organizes multi-source similarities into an evidence graph. At the selected graph setting, it identifies 244 Campaign candidates with 97.12% multi-sample coverage and 97.68% edge family homogeneity. The graph sensitivity analysis shows the expected trade-off: looser rules retain broader coverage, while stricter rules improve homogeneity and agreement with family labels. These communities are not direct judgments of real-world attacker organizations. Instead, they group samples with similar static evidence so that analysts can prioritize them for further inspection. The report generation module follows the same principle. deepseek-v4-flash receives only privacy-preserving aggregated evidence, and each generated claim must cite the local evidence catalog. All 120 LLM-generated multi-sample reports and 124 template-generated singleton reports achieve 100% on the machine-checkable verification metrics. Taken together, EVIDENCEMALGRAPH turns family attribution from an isolated classification result into an inspectable workflow in which predictions, cross-sample relationships, and report statements remain linked to APK-derived evidence. By maintaining this link throughout the analysis, EVIDENCEMALGRAPH provides a reproducible basis for analyst-facing Android malware investigation.

CRedit authorship contribution statement

Shishuai Yang: Conceptualization, Methodology, Software, Data curation, Formal analysis, Investigation, Validation, Visualization, Funding acquisition, Writing – original draft; **Xiaoyu Li:** Methodology, Software, Formal analysis, Validation, Writing – review & editing; **Kailun Yan:** Methodology, Formal analysis, Validation, Writing – review & editing; **Chao Wang:** Data curation, Investigation, Validation, Writing – review & editing; **Weixiang Ren:** Conceptualization, Supervision, Project

administration, Funding acquisition, Writing – review & editing; **Yan-gong Chen**: Investigation, Validation, Visualization, Writing – review & editing; **Kebao Chen**: Software, Data curation, Investigation, Validation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was primarily supported by the Natural Science Foundation of Henan Province (Grant Nos. 262300420296 and 252300423468), and was also supported by the Youth Research Funds Plan of Zhengzhou University of Aeronautics (Grant Nos. 26ZHQN01023 and 26ZHQN01025) and the Research Programs of Henan Science and Technology Department (Grant No. 262102210027).

Data availability

Data will be made available on request.

References

- Aafer, Y., Du, W., & Yin, H. (2013). DroidAPIMiner: Mining API-level features for robust malware detection in Android. In *Proceedings of the 9th International conference on security and privacy in communication networks (securecomm)*, Sydney, NSW, Australia, September 25–28, 2013.
- Arp, D., Spreitzenbarth, M., Hubner, M., Gascon, H., & Rieck, K. (2014). DREBIN: Effective and explainable detection of Android malware in your pocket. In *Proceedings of the 21st Annual network and distributed system security symposium (NDSS)*, San Diego, CA, USA, February 23–26, 2014.
- Bayer, U., Comparetti, P. M., Hlauschek, C., Krügel, C., & Kirda, E. (2009). Scalable, behavior-based malware clustering. In *Proceedings of the network and distributed system security symposium (NDSS)*, San Diego, California, USA, February 8–11, 2009.
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment*, 2008 (10), P10008.
- DeepSeek-AI (2026). DeepSeek-V4-flash. https://api-docs.deepseek.com/quick_start/pricing.
- Dhuliawala, S., Komeili, M., Xu, J., Raileanu, R., Li, X., Celikyilmaz, A., & Weston, J. (2024). Chain-of-verification reduces hallucination in large language models. In *Findings of the 62nd Annual meeting of the association for computational linguistics (ACL)*, Bangkok, Thailand, August 11–16, 2024.
- Frenklach, T., Cohen, D., Shabtai, A., & Puzis, R. (2021). Android malware detection via an app similarity graph. *Computers & Security*, 109, 102386.
- Gao, H., Cheng, S., & Zhang, W. (2021). GDroid: Android malware detection and classification with graph convolutional network. *Computers & Security*, 106, 102264.
- Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. In *Proceedings of the 28th Conference on empirical methods in natural language processing (EMNLP)*, Singapore, December 6–10, 2023.
- Github (2017). Androguard. <https://github.com/androguard/androguard>.
- Google (2025a). Application fundamentals. <https://developer.android.com/guide/components/fundamentals>.
- Google (2025b). Dalvik executable format. <https://source.android.com/docs/core/runtime/dex-format>.
- Google (2026a). AccessibilityService. <https://developer.android.com/reference/android/accessibilityservice/AccessibilityService>.
- Google (2026b). App manifest overview. <https://developer.android.com/guide/topics/manifest/manifest-intro>.
- Google (2026c). App resources overview. <https://developer.android.com/guide/topics/resources/providing-resources>.
- Google (2026d). Manifest.permission. https://developer.android.com/reference/android/Manifest.permission#SYSTEM_ALERT_WINDOW.
- Google (2026e). Permissions on Android. <https://developer.android.com/guide/topics/permissions/overview>.
- Google (2026f). Sign your app. <https://developer.android.com/studio/publish/app-signing>.
- Google (2026g). SmsManager. <https://developer.android.com/reference/android/telephony/SmsManager>.
- Hou, S., Ye, Y., Song, Y., & Abdulhayoglu, M. (2017). HinDroid: An intelligent Android malware detection system based on structured heterogeneous information network. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining (kdd)*, Halifax, NS, Canada, August 13–17, 2017.
- Hurier, M., Suarez-Tangil, G., Dash, S. K., Bissyand'e, T. F., Traon, Y. L., Klein, J., & Cavallaro, L. (2017). Euphony: Harmonious unification of cacophonous anti-virus vendor labels for Android malware. In *Proceedings of the 14th International conference on mining software repositories (MSR)*, Buenos Aires, Argentina, May 20–28, 2017.
- IBM (2026). What is Jaccard similarity? <https://www.ibm.com/think/topics/jaccard-similarity>.
- Imtiaz, S. I., Rehman, S. u., Javed, A. R., Jalil, Z., Liu, X., & Alnumay, W. S. (2021). DeepAMD: Detection and identification of Android malware using high-efficient deep artificial neural network. *Future Generation Computer Systems*, 115, 844–856.
- Jang, J., Brumley, D., & Venkataraman, S. (2011). BitShred: Feature hashing malware for scalable triage and semantic analysis. In *Proceedings of the 18th ACM conference on computer and communications security (CCS)*, Chicago, Illinois, USA, October 17–21, 2011.
- Joyce, R. J., Patel, T., Nicholas, C., & Raff, E. (2023). AVScan2Vec: Feature learning on antivirus scan data for production-scale malware corpora. In *Proceedings of the 16th ACM workshop on artificial intelligence and security (aisec@ccs)*, Copenhagen, Denmark, November 30, 2023.
- Karbab, E. B., Debbabi, M., Derhab, A., & Mouheb, D. (2018). MalDozer: Automatic framework for Android malware detection using deep learning. *Digital Investigation*, 24, S48–S59.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th Annual conference on neural information processing systems (NeurIPS)*, virtual event, December 6–12, 2020.
- Li, Y., Jang, J., Hu, X., & Ou, X. (2017). Android malware clustering through malicious payload mining. In *Proceedings of the 20th International symposium on research in attacks, intrusions and defenses (RAID)*, Atlanta, GA, USA, September 18–20, 2017.
- Li, Y., Yuan, D., Zhang, T., Cai, H., Lo, D., Gao, C., Luo, X., & Jiang, H. (2024). Meta-learning for multi-family Android malware classification. *ACM Transactions on Software Engineering and Methodology*, 33 (7), 174:1–174:27.
- Lundberg, S. M., & Lee, S. (2017). A unified approach to interpreting model predictions. In *Proceedings of the 31st Annual conference on neural information processing systems (NIPS)*, long beach, CA, USA, December–9, 2017.
- Manakul, P., Liusie, A., & Gales, M. J. F. (2023). SelfCheckGPT: Zero-resource black-box hallucination detection for generative large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing (EMNLP)*, Singapore, December 6–10, 2023 (pp. 9004–9017).
- Mariconti, E., Onwuzurike, L., Andriotis, P., De Cristofaro, E., Ross, G. J., & Stringhini, G. (2017). MaMaDroid: Detecting Android malware by building markov chains of behavioral models. In *Proceedings of the 24th annual network and distributed system security symposium (NDSS)*, San Diego, CA, USA, February 26–March 1, 2017.
- Microsoft (2025). How Microsoft names Malware. <https://learn.microsoft.com/en-us/unified-secpops/malware-naming>.
- Narayanan, A., Chandramohan, M., Chen, L., & Liu, Y. (2018). A multi-view context-aware approach to Android malware detection and malicious code localization. *Empirical Software Engineering*, 23 (3), 1222–1274.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why should i trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining (kdd)*, San Francisco, CA, USA, August 13–17, 2016.
- Rieck, K., Holz, T., Willems, C., D'ussel, P., & Laskov, P. (2008). Learning and classification of malware behavior. In *Proceedings of the 5th International conference on detection of intrusions and malware, and vulnerability assessment (DIMVA)*, Paris, France, July 10–11, 2008.
- Sebastian, M., Rivera, R., Kotzias, P., & Caballero, J. (2016). AVClass: A tool for massive malware labeling. In *Proceedings of the 19th International symposium on research in attacks, intrusions and defenses (RAID)*, Paris, France, September 19–21, 2016.
- Sebastian, S., & Caballero, J. (2020). AVclass2: Massive malware tag extraction from AV labels. In *Proceedings of the 36th Annual computer security applications conference (ACSAC)*, virtual event / austin, TX, USA, December 7–11, 2020.
- StatCounter (2026). Mobile operating system market share worldwide.
- Wang, L., Wang, H., He, R., Tao, R., Meng, G., Luo, X., & Liu, X. (2022). MalRadar: Demystifying Android malware in the new era. In *Proceedings of the 48th ACM SIGMETRICS/IFIP PERFORMANCE joint international conference on measurement and modeling of computer systems (sigmetrics/performance)*, Mumbai, India, June 6–10, 2022.
- Ying, Z., Bourgeois, D., You, J., Zitnik, M., & Leskovec, J. (2019). GNNExplainer: Generating explanations for graph neural networks. In *Proceedings of the 33rd Annual conference on neural information processing systems (neurIPS)*, Vancouver, BC, Canada, December 8–14, 2019.
- Zhang, M., Duan, Y., Yin, H., & Zhao, Z. (2014). Semantics-aware Android malware classification using weighted contextual API dependency graphs. In *Proceedings of the 21st ACM conference on computer and communications security (CCS)*, Scottsdale, AZ, USA, November 3–7, 2014 (pp. 1105–1116).